Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп’ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 5

Тема: Обробка текстових даних, рядкові методи та регулярні вирази

Мета роботи: Опанувати практичні навички алгоритмічної обробки, валідації, нормалізації та структурування текстової інформації мовою Python. Навчитися застосовувати базові та розширені методи рядкового типу str для очищення й токенізації даних, проектувати шаблони динамічного форматування за допомогою сучасних f-рядків (f-strings) із точним позиціонуванням ширини полів та числових специфікаторів, а також освоїти математичний апарат регулярних виразів (модуль re) для синтаксичного аналізу, валідації та вилучення структурованих параметрів із журналів подій і конфігураційних файлів комп'ютерних систем.

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп’ютерна інженерія»
Освітня програма Комп’ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи та постановка індивідуального завдання

Мета роботи: опанувати алгоритмічну обробку, валідацію, нормалізацію та структурування текстових даних мовою Python; застосувати методи типу str для очищення й токенізації рядків, f-рядки для форматованого виведення та модуль re з іменованими групами захоплення для розбору системних журналів.

1.1. Індивідуальне завдання (варіант 2)

Параметр	**Значення**
Тип журналу	системний журнал Linux Syslog
Поля кадру	Month Day Time, Hostname, Daemon_Name[PID], Severity (INFO/ERR), Message
Критерії	виділення процесів із помилками ERR, групування за Daemon_Name

1.2. Структура кадру журналу

Кожен рядок журналу описує одну подію та має такий формат:

Mon DD HH:MM:SS hostname daemon[PID]: SEVERITY: message

Приклад: Sep 2 08:14:21 srv-web01 sshd[1034]: ERR: Failed password for invalid user root from 203.0.113.45 port 41872. Класичний syslog не містить окремого поля рівня важливості в такому вигляді, тому в роботі прийнято, що повідомлення починається з маркера INFO: або ERR:, як вимагає завдання. Місяць позначається трилітерним скороченням, день може бути одно- або двоцифровим (для дня з однієї цифри syslog додає додатковий пробіл), PID записується в квадратних дужках одразу після назви демона.

Вхідний файл input_logs.txt містить 23 рядки: 17 коректних записів від п’яти демонів (sshd, nginx, postgres, cron, systemd) на двох хостах, три службові рядки (два коментарі й один порожній) та три навмисно некоректні рядки для перевірки обробки помилок. Деякі коректні рядки містять зайві пробіли,

табуляцію та нерозривний пробіл U+00A0, щоб продемонструвати роботу очищення.

2. Проєктування регулярного виразу

Скомпільований регулярний вираз LOG_PATTERN записано в режимі re.VERBOSE, що дозволяє розбити шаблон на рядки й додати коментарі:

```
^(?P<month>[A-Z][a-z]{2})\s+
(?P<day>\d{1,2})\s+
(?P<time>\d{2}:\d{2}:\d{2})\s+
(?P<hostname>[A-Za-z0-9][A-Za-z0-9.\-]*)\s+
(?P<daemon>[A-Za-z0-9_.\-/]+)
\[(?P<pid>\d+)\]:\s+
(?P<severity>INFO|ERR):\s+
(?P<message>.+)$
```

Розшифровка фрагментів шаблону та іменованих груп захоплення:

Фрагмент	**Призначення**
^ та $	прив’язка до початку та кінця нормалізованого рядка: сторонній текст до чи після кадру не допускається
(?P<month>[A-Z][a-z]{2})	група month: велика латинська літера й дві малі (Sep, Oct)
\s+	один або більше пробільних символів; шаблон витримує різну кількість пробілів між полями
(?P<day>\d{1,2})	група day: одна або дві цифри дня місяця
(?P<time>\d{2}:\d{2}:\d{2})	група time: час у форматі HH:MM:SS
(?P<hostname>[A-Za-z0-9][A-Za-z0-9.\-]*)	група hostname: починається з літери чи цифри, далі літери, цифри, крапки та дефіси
(?P<daemon>[A-Za-z0-9_.\-/]+)	група daemon: назва процесу; символ [у клас не входить, тому межа з PID однозначна
\[(?P<pid>\d+)\]:	група pid: одна або більше цифр у екранованих квадратних дужках, за якими стоїть двокрапка
(?P<severity>INFO|ERR):	група severity: альтернація двох допустимих рівнів; інші рівні (WARN тощо) не проходять
(?P<message>.+)$	група message: решта рядка до кінця
re.VERBOSE	пробіли поза класами символів та коментарі після # ігноруються рушієм

Шаблон не містить вкладених квантифікаторів виду (a+)+, а кожне поле відокремлене від сусіднього літералом або пробільним символом, якого немає

в класі попереднього поля. Тому на кожній позиції рушій має не більше однієї осмисленої гілки розбору, і зіставлення виконується за лінійний час O(N) без ризику катастрофічного повернення. Компіляція виконується один раз під час імпорту модуля, а не при кожному рядку журналу.

Іменована група (?P<name>...) дозволяє отримати результат у вигляді словника: match.groupdict() повертає {"month": "Sep", "day": "2", "time": "08:14:21", "hostname": "srv-web01", "daemon": "sshd", "pid": "1034", "severity": "ERR", "message": "..."}. Усі значення є рядками, тому числові поля day і pid перетворюються на int вже після валідації.

3. Алгоритм обробки та валідація

Обробка журналу виконується конвеєром із таких етапів:

1) відкрити файл і зчитувати його рядок за рядком (ітератор файлу не завантажує весь файл у пам’ять);
2) очистити рядок: видалити керуючі символи, замінити будь-які послідовності пробільних символів одним пробілом (" ".join(text.split())) та прибрати крайні пробіли;
3) пропустити порожні рядки й коментарі, що починаються з #;
4) застосувати LOG_PATTERN.search(); якщо збігу немає, рядок додається до списку відхилених з причиною «не відповідає формату кадру»;
5) виконати семантичну валідацію значень (таблиця нижче); при невдачі рядок відхиляється з конкретною причиною;
6) перетворити day і pid на int, зберегти запис разом з номером рядка;
7) згрупувати записи за назвою демона, підрахувати INFO, ERR і множину PID із помилками;
8) вивести таблицю записів, таблицю групування, список відхилених рядків і підсумкову статистику.

Поле	Правило валідації
month	належить множині {Jan, Feb, …, Dec}
day	1 ≤ day ≤ кількість днів у місяці (для лютого — 29)
time	0 ≤ HH ≤ 23, 0 ≤ MM ≤ 59, 0 ≤ SS ≤ 59
pid	1 ≤ PID ≤ 4 194 304 (типове значення kernel.pid_max для 64-бітних систем)
severity	гарантовано INFO або ERR самим регулярним виразом

Часова складність обробки становить O(N · L), де N — кількість рядків, L — середня довжина рядка, оскільки кожен рядок проходить лінійні етапи очищення, зіставлення та валідації. Групування виконується за O(N) завдяки хеш-таблиці (словнику) з доступом за ключем у середньому за O(1). Метод str.join() об'єднує лексеми за лінійний час, на відміну від повторної конкатенації через +=, яка мала б квадратичну складність.

4. Програмний код

4.1. Файл text_sanitizer.py

```
"""
Лабораторна робота № 5. Модуль 1. Варіант 2.
Базове очищення та нормалізація рядків системного журналу Linux Syslog
методами типу str та простим регулярним виразом.
Освітній компонент: Програмування на Python.
"""

import re

# Керуючі символи ASCII, окрім \t (0x09) та \n (0x0A): NUL, BEL, BS, VT, FF, ESC,
DEL тощо
_CONTROL_CHARS = re.compile(r"[\x00-\x08\x0b-\x1f\x7f]")


def remove_control_chars(text: str) -> str:
    """Видаляє некоректні (керуючі) символи, що не несуть корисної інформації."""
    return _CONTROL_CHARS.sub("", text)


def collapse_whitespace(text: str) -> str:
    """
    Замінює будь-яку послідовність пробільних символів (пробіли, табуляції,
    нерозривні пробіли U+00A0) одним пробілом і прибирає крайні пробіли.
```

```
    split() без аргументів розбиває рядок за будь-яким пробільним символом
    і відкидає порожні лексеми, а join() збирає їх назад за O(N).
    """
    return " ".join(text.split())


def sanitize_log_line(raw_line: str) -> str:
    """Повний цикл очищення: керуючі символи -> нормалізація пробілів."""
    return collapse_whitespace(remove_control_chars(raw_line))


def main() -> None:
    """Невеликий самотест модуля на брудних рядках."""
    samples = [
        "  Sep  2\t08:15:02   srv-web01 nginx[2210]:   INFO: worker started \n",
        "Sep  2 08:18:00 srv-web01 cron[877]: INFO: (root)
\u00a0CMD\u00a0(backup.sh)",
        "Sep  3 00:05:44 srv-web01 systemd[1]:\x07 ERR:\x00 service failed\r\n",
        "   \t  \n",
    ]
    print("=" * 78)
    print(f"{'ТЕСТ МОДУЛЯ ОЧИЩЕННЯ ТЕКСТУ':^78}")
    print("=" * 78)
    for idx, raw in enumerate(samples, start=1):
        clean = sanitize_log_line(raw)
        print(f"{idx}) до:    {raw!r}")
        print(f"   після: {clean!r}")
    print("=" * 78)


if __name__ == "__main__":
    main()
```

4.2. Файл log_parser.py

```
"""
Лабораторна робота № 5. Модуль 2. Варіант 2.
Синтаксичний розбір системного журналу Linux Syslog регулярними виразами
з іменованими групами, валідація полів, групування за демоном (Daemon_Name)
та формування табличного звіту через f-strings.

Формат кадру:
    Mon DD HH:MM:SS hostname daemon[PID]: SEVERITY: message
Приклад:
    Sep  2 08:14:21 srv-web01 sshd[1034]: ERR: Failed password for root
Освітній компонент: Програмування на Python.
"""
```

```python
import re
import time
from pathlib import Path

from text_sanitizer import sanitize_log_line

# Скомпільований регулярний вираз з іменованими групами (режим VERBOSE дозволяє коментарі)
LOG_PATTERN = re.compile(
    r"""
    ^(?P<month>[A-Z][a-z]{2})\s+          # місяць: три літери (Sep)
    (?P<day>\d{1,2})\s+                   # день місяця: 1-2 цифри
    (?P<time>\d{2}:\d{2}:\d{2})\s+        # час HH:MM:SS
    (?P<hostname>[A-Za-z0-9][A-Za-z0-9.\-]*)\s+   # ім'я хоста
    (?P<daemon>[A-Za-z0-9_.\-/]+)         # назва демона
    \[(?P<pid>\d+)\]:\s+                  # PID у квадратних дужках
    (?P<severity>INFO|ERR):\s+            # рівень важливості
    (?P<message>.+)$                      # текст повідомлення до кінця рядка
    """,
    re.VERBOSE,
)

# Максимальна кількість днів у місяці (лютий беремо з урахуванням високосного року)
MONTH_DAYS: dict[str, int] = {
    "Jan": 31, "Feb": 29, "Mar": 31, "Apr": 30, "May": 31, "Jun": 30,
    "Jul": 31, "Aug": 31, "Sep": 30, "Oct": 31, "Nov": 30, "Dec": 31,
}
PID_MAX: int = 4_194_304   # типове значення kernel.pid_max для 64-бітних систем

REPORT_WIDTH: int = 88


def validate_record(record: dict) -> str | None:
    """
    Семантична валідація вилучених полів.
    Повертає текст причини відхилення або None, якщо запис коректний.
    """
    month, day = record["month"], int(record["day"])
    if month not in MONTH_DAYS:
        return f"невідомий місяць '{month}'"
    if not 1 <= day <= MONTH_DAYS[month]:
        return f"день {day} поза діапазоном для {month}"

    hours, minutes, seconds = (int(part) for part in record["time"].split(":"))
    if hours > 23 or minutes > 59 or seconds > 59:
        return f"некоректний час {record['time']}"

    if not 1 <= int(record["pid"]) <= PID_MAX:
        return f"PID {record['pid']} поза діапазоном 1..{PID_MAX}"
```

```
    return None


def parse_syslog_file(file_path: Path) -> tuple[list[dict], list[tuple[int, str,
str]]]:
    """
    Зчитує журнал і повертає кортеж (коректні_записи, відхилені_рядки).
    Відхилений рядок - це кортеж (номер_рядка, причина, нормалізований_текст).
    """
    if not file_path.exists():
        raise FileNotFoundError(f"Вхідний файл журналу '{file_path}' не
знайдено.")

    records: list[dict] = []
    rejected: list[tuple[int, str, str]] = []

    with open(file_path, mode="r", encoding="utf-8") as f:
        for line_num, line in enumerate(f, start=1):
            normalized = sanitize_log_line(line)
            if not normalized or normalized.startswith("#"):
                continue  # порожні рядки та коментарі

            match = LOG_PATTERN.search(normalized)
            if match is None:
                rejected.append((line_num, "не відповідає формату кадру",
normalized))
                continue

            record = match.groupdict()
            reason = validate_record(record)
            if reason is not None:
                rejected.append((line_num, reason, normalized))
                continue

            record["day"] = int(record["day"])
            record["pid"] = int(record["pid"])
            record["line_num"] = line_num
            records.append(record)

    return records, rejected


def group_by_daemon(records: list[dict]) -> dict[str, dict]:
    """
    Групує записи за назвою демона. Для кожного демона підраховує
    загальну кількість подій, INFO, ERR та множину PID із помилками.
    """
    groups: dict[str, dict] = {}
    for rec in records:
        g = groups.setdefault(rec["daemon"], {"total": 0, "info": 0, "err": 0,
"err_pids": set()})
```

```
        g["total"] += 1
        if rec["severity"] == "ERR":
            g["err"] += 1
            g["err_pids"].add(rec["pid"])
        else:
            g["info"] += 1
    return groups


def shorten(text: str, width: int) -> str:
    """Обрізає текст до width символів, додаючи символ «…» при скороченні."""
    return text if len(text) <= width else text[: width - 1] + "…"


def print_records_table(records: list[dict]) -> None:
    """Виводить таблицю коректно розібраних записів."""
    print("=" * REPORT_WIDTH)
    print(f"{'ЗВІТ ПРО РОЗБІР ЖУРНАЛУ LINUX SYSLOG':^{REPORT_WIDTH}}")
    print("=" * REPORT_WIDTH)
    print(f"| {'Час':^15} | {'Хост':^10} | {'Процес[PID]':^14} | {'Рівень':^6} |
{'Повідомлення':^27} |")
    print("-" * REPORT_WIDTH)
    for rec in records:
        stamp = f"{rec['month']} {rec['day']:>2d} {rec['time']}"
        proc = f"{rec['daemon']}[{rec['pid']}]"
        print(
            f"| {stamp:<15} | {rec['hostname']:<10} | {proc:<14} |
{rec['severity']:^6} | "
            f"{shorten(rec['message'], 27):<27} |"
        )
    print("=" * REPORT_WIDTH)


def print_group_table(groups: dict[str, dict]) -> None:
    """Виводить таблицю групування за Daemon_Name, відсортовану за кількістю
помилок."""
    print(f"\n{'ГРУПУВАННЯ ПОДІЙ ЗА ДЕМОНОМ (Daemon_Name)':^{REPORT_WIDTH}}")
    print("-" * REPORT_WIDTH)
    print(f"| {'Демон':^12} | {'Всього':^6} | {'INFO':^5} | {'ERR':^5} | {'%
ERR':^7} | {'PID із помилками':^34} |")
    print("-" * REPORT_WIDTH)
    ordered = sorted(groups.items(), key=lambda item: (-item[1]["err"], item[0]))
    for daemon, g in ordered:
        share = 100.0 * g["err"] / g["total"]
        pids = ", ".join(str(p) for p in sorted(g["err_pids"])) or "-"
        print(
            f"| {daemon:<12} | {g['total']:>6d} | {g['info']:>5d} |
{g['err']:>5d} | "
            f"{share:>6.1f}% | {pids:<34} |"
        )
    print("-" * REPORT_WIDTH)
```

```python
def print_rejected(rejected: list[tuple[int, str, str]]) -> None:
    """Виводить список відхилених рядків із причинами."""
    print(f"\n{'ВІДХИЛЕНІ РЯДКИ':^{REPORT_WIDTH}}")
    print("-" * REPORT_WIDTH)
    if not rejected:
        print("  Відхилених рядків немає.")
    for line_num, reason, text in rejected:
        print(f"  Рядок {line_num:>2d}: {reason}")
        print(f"            {shorten(text, REPORT_WIDTH - 12)}")
    print("-" * REPORT_WIDTH)


def print_summary(records: list[dict], rejected: list, groups: dict[str, dict])
-> None:
    """Виводить підсумкову агреговану статистику."""
    err_total = sum(g["err"] for g in groups.values())
    err_daemons = [name for name, g in groups.items() if g["err"] > 0]
    worst = max(groups.items(), key=lambda item: (item[1]["err"], item[0]))
    share = 100.0 * err_total / len(records) if records else 0.0

    print(f"\n{'ПІДСУМКОВА СТАТИСТИКА':^{REPORT_WIDTH}}")
    print("-" * REPORT_WIDTH)
    print(f"  Коректно розібрано записів:       {len(records):>4d}")
    print(f"  Відхилено рядків:                 {len(rejected):>4d}")
    print(f"  Унікальних демонів:               {len(groups):>4d}")
    print(f"  Подій рівня ERR:                  {err_total:>4d} ({share:.1f}%
від усіх записів)")
    print(f"  Демонів, що мали помилки:         {len(err_daemons):>4d}")
    print(f"  Найбільше помилок у демона:       {worst[0]} ({worst[1]['err']})")
    print("=" * REPORT_WIDTH + "\n")


def main() -> None:
    """Головна точка входу модуля розбору журналу."""
    log_path = Path(__file__).with_name("input_logs.txt")

    t_start = time.perf_counter()
    records, rejected = parse_syslog_file(log_path)
    groups = group_by_daemon(records)

    print()
    print_records_table(records)
    print_group_table(groups)
    print_rejected(rejected)
    print_summary(records, rejected, groups)
    elapsed = time.perf_counter() - t_start

    print(f"Час виконання повного циклу аналізу тексту: {elapsed * 1000:.3f}
мс\n")
```

```
if __name__ == "__main__":
    main()
```

5. Результати тестування

Запуск виконувався в середовищі Conda ce_lab_env командами python text_sanitizer.py та python log_parser.py.

5.1. Вхідний файл input_logs.txt

Фрагмент системного журналу Linux Syslog (вибірка з двох хостів)
Sep 2 08:14:03 srv-web01 sshd[1021]: INFO: Accepted publickey for admin from 192.168.1.15 port 50122 ssh2
Sep 2 08:14:21 srv-web01 sshd[1034]: ERR: Failed password for invalid user root from 203.0.113.45 port 41872
Sep 2 08:14:23 srv-web01 sshd[1034]: ERR: Failed password for invalid user admin from 203.0.113.45 port 41874
Sep 208:15:02 srv-web01 nginx[2210]: INFO: worker process 2214 started
Sep 2 08:15:47 srv-web01 nginx[2210]: ERR: connect() failed (111: Connection refused) while connecting to upstream
Sep 2 08:16:10 srv-db02 postgres[3105]: INFO: database system is ready to accept connections
Sep 2 08:17:33 srv-db02 postgres[3118]: ERR: could not write to file "pg_wal/000000010000000000000003": No space left on device
Sep 2 08:17:34 srv-db02 postgres[3118]: ERR: checkpoint request failed
Sep 2 08:18:00 srv-web01 cron[877]: INFO: (root) CMD (/usr/local/bin/backup.sh)
Sep 2 08:19:12 srv-web01 systemd[1]: INFO: Started Daily apt download activities.

Sep 2 08:21:09 srv-db02 sshd[4410]: ERR: maximum authentication attempts exceeded for postgres from 198.51.100.7 port 52114
Sep 2 08:21:30 srv-web01 nginx[2210]: ERR: open() "/var/www/html/favicon.ico" failed (2: No such file or directory)
Sep 2 08:22:02 srv-web01 sshd[1021]: INFO: Disconnected from user admin 192.168.1.15 port 50122
Sep 3 00:00:01 srv-db02 cron[912]: INFO: (postgres) CMD (vacuumdb --all)
Sep 3 00:03:18 srv-db02 postgres[3200]: INFO: autovacuum launched for database "appdb"

Sep 3 00:05:44 srv-web01 systemd[1]: ERR: nginx.service: Main process exited, code=exited, status=1/FAILURE
Sep 3 00:05:45 srv-web01 systemd[1]: INFO: Starting A high performance web server and a reverse proxy server...
Нижче навмисно некоректні рядки для перевірки обробки помилок
Sep 3 09:15:44 srv-web01 kernel: ERR: EXT4-fs error (device sda1): unable to read inode block
Sep 3 25:61:00 srv-web01 cron[901]: INFO: (root) CMD (/usr/lib/php/sessionclean)
Sep 3 09:20:10 srv-web01 sshd[1040]: WARN: Connection closed by 10.0.0.9 [preauth]

У рядках 3, 5, 7 та 10 навмисно залишено зайві пробіли, табуляцію та нерозривний пробіл U+00A0; у файлі вони не відрізняються візуально від звичайних пробілів, але модуль очищення нормалізує їх.

5.2. Результат роботи text_sanitizer.py

```
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab5\lab05> conda activate ce_lab_env
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab5\lab05> & C:\Users\Janis\miniconda3\envs\ce_lab_env\python.exe "c:/Users/Janis/Desktop/ypok/Програмування на Python. Базовий рівень/lab5/lab05/text_sanitizer.py"
================================================================================
                        ТЕСТ МОДУЛЯ ОЧИЩЕННЯ ТЕКСТУ
================================================================================
1) до:    '  Sep  2\t08:15:02   srv-web01 nginx[2210]:   INFO: worker started \n'
   після: 'Sep 2 08:15:02 srv-web01 nginx[2210]: INFO: worker started'
2) до:    'Sep  2 08:18:00 srv-web01 cron[877]: INFO: (root)\xa0CMD\xa0(backup.sh)'
   після: 'Sep 2 08:18:00 srv-web01 cron[877]: INFO: (root) CMD (backup.sh)'
3) до:    'Sep  3 00:05:44 srv-web01 systemd[1]:\x07 ERR:\x00 service failed\r\n'
   після: 'Sep 3 00:05:44 srv-web01 systemd[1]: ERR: service failed'
4) до:    '   \t  \n'
   після: ''
================================================================================
```

Скріншот 1 — Самотест модуля очищення тексту

Приклад 1 демонструє заміну табуляції та кількох пробілів одним; у прикладі 2 нерозривні пробіли замінено звичайними; у прикладі 3 видалено керуючі символи BEL (\x07) та NUL (\x00) і завершальні \r\n; у прикладі 4 рядок із самих пробілів перетворюється на порожній і далі пропускається.

5.3. Результат роботи log_parser.py

```
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab5\lab05> conda activate ce_lab_env
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab5\lab05> & C:\Users\Janis\miniconda3\envs\c
e_lab_env\python.exe "c:/Users/Janis/Desktop/ypok/Програмування на Python. Базовий рівень/lab5/lab05/log_parser.py"

================================================================================
                        ЗВІТ ПРО РОЗБІР ЖУРНАЛУ LINUX SYSLOG
================================================================================
|       Час       |    Хост    |  Процес[PID]   | Рівень |        Повідомлення        |
--------------------------------------------------------------------------------
| Sep  2 08:14:03 | srv-web01  | sshd[1021]     |  INFO  | Accepted publickey for adm… |
| Sep  2 08:14:21 | srv-web01  | sshd[1034]     |  ERR   | Failed password for invali… |
| Sep  2 08:14:23 | srv-web01  | sshd[1034]     |  ERR   | Failed password for invali… |
| Sep  2 08:15:02 | srv-web01  | nginx[2210]    |  INFO  | worker process 2214 started |
| Sep  2 08:15:47 | srv-web01  | nginx[2210]    |  ERR   | connect() failed (111: Con… |
| Sep  2 08:16:10 | srv-db02   | postgres[3105] |  INFO  | database system is ready t… |
| Sep  2 08:17:33 | srv-db02   | postgres[3118] |  ERR   | could not write to file "p… |
| Sep  2 08:17:34 | srv-db02   | postgres[3118] |  ERR   | checkpoint request failed   |
| Sep  2 08:18:00 | srv-web01  | cron[877]      |  INFO  | (root) CMD (/usr/local/bin… |
| Sep  2 08:19:12 | srv-web01  | systemd[1]     |  INFO  | Started Daily apt download… |
| Sep  2 08:21:09 | srv-db02   | sshd[4410]     |  ERR   | maximum authentication att… |
| Sep  2 08:21:30 | srv-web01  | nginx[2210]    |  ERR   | open() "/var/www/html/favi… |
| Sep  2 08:22:02 | srv-web01  | sshd[1021]     |  INFO  | Disconnected from user adm… |
| Sep  3 00:00:01 | srv-db02   | cron[912]      |  INFO  | (postgres) CMD (vacuumdb -… |
| Sep  3 00:03:18 | srv-db02   | postgres[3200] |  INFO  | autovacuum launched for da… |
| Sep  3 00:05:44 | srv-web01  | systemd[1]     |  ERR   | nginx.service: Main proces… |
| Sep  3 00:05:45 | srv-web01  | systemd[1]     |  INFO  | Starting A high performanc… |
================================================================================

                    ГРУПУВАННЯ ПОДІЙ ЗА ДЕМОНОМ (Daemon_Name)
--------------------------------------------------------------------------------
|    Демон     | Всього | INFO |  ERR  |  % ERR  |          PID із помилками          |
--------------------------------------------------------------------------------
| sshd         |      5 |    2 |     3 |   60.0% | 1034, 4410                         |
| nginx        |      3 |    1 |     2 |   66.7% | 2210                               |
| postgres     |      4 |    2 |     2 |   50.0% | 3118                               |
| systemd      |      3 |    2 |     1 |   33.3% | 1                                  |
| cron         |      2 |    2 |     0 |    0.0% | -                                  |
--------------------------------------------------------------------------------

                                ВІДХИЛЕНІ РЯДКИ
--------------------------------------------------------------------------------
  Рядок 21: не відповідає формату кадру
            Sep 3 09:15:44 srv-web01 kernel: ERR: EXT4-fs error (device sda1): unable t…
  Рядок 22: некоректний час 25:61:00
            Sep 3 25:61:00 srv-web01 cron[901]: INFO: (root) CMD (/usr/lib/php/sessionc…
  Рядок 23: не відповідає формату кадру
            Sep 3 09:20:10 srv-web01 sshd[1040]: WARN: Connection closed by 10.0.0.9 [p…
--------------------------------------------------------------------------------

                            ПІДСУМКОВА СТАТИСТИКА
--------------------------------------------------------------------------------
  Коректно розібрано записів:         17
  Відхилено рядків:                    3
  Унікальних демонів:                  5
  Подій рівня ERR:                     8 (47.1% від усіх записів)
  Демонів, що мали помилки:            4
  Найбільше помилок у демона:       sshd (3)
================================================================================
```

Скріншот 2— Звіт про розбір журналу, групування за демоном і підсумкова статистика

Перевірка результатів. З 23 рядків файлу три є службовими (два коментарі та один порожній рядок), решта 20 проходять через регулярний вираз: 17 розібрано коректно й 3 відхилено. Відхилення мають різні причини:

рядок 21 не містить PID (kernel: замість kernel[PID]:), рядок 22 містить неіснуючий час 25:61:00 і відхиляється вже валідатором, а рядок 23 має рівень WARN, якого шаблон не допускає.

Групування за демоном: sshd — 5 подій (3 ERR, PID 1034 та 4410), nginx — 3 події (2 ERR, PID 2210), postgres — 4 події (2 ERR, PID 3118), systemd — 3 події (1 ERR, PID 1), cron — 2 події без помилок. Сума ERR становить 3 + 2 + 2 + 1 = 8 з 17 записів, тобто 47,1 %. Найбільше помилок має sshd (3), найвища частка помилок — у nginx (66,7 %). Процеси з помилками виділено окремим стовпцем «PID із помилками».

6. Посилання на віддалений репозиторій

Профіль: https://github.com/kefir4ikk

Репозиторій лабораторної роботи : https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab5

7. Висновки

У ході лабораторної роботи реалізовано розбір системного журналу Linux Syslog для варіанта 2. Рядки в Python є незмінними послідовностями Unicode, тому очищення будується на створенні нових рядків: комбінація split() та join() замінила будь-які послідовності пробільних символів одним пробілом за лінійний час і привела «брудні» рядки з табуляціями та нерозривними пробілами до єдиного вигляду.

Скомпільований регулярний вираз з іменованими групами дозволив вилучити всі п’ять обов’язкових полів кадру в один прохід і одразу отримати результат у вигляді словника через groupdict(). Шаблон не містить неоднозначних вкладених квантифікаторів, тому його складність лінійна, а ризик катастрофічного повернення відсутній. Окремий етап семантичної валідації (місяць, день, час, PID) виявив некоректний час 25:61:00, який

регулярний вираз за синтаксисом пропускає, що підтверджує необхідність двох рівнів перевірки: синтаксичного та семантичного.

Групування за назвою демона через словник дало агреговану картину: з 17 записів 8 мають рівень ERR (47,1 %), найбільше помилок у sshd, найвища частка помилок у nginx. Форматований вивід через f-рядки зі специфікаторами ширини та вирівнювання забезпечив зручну для читання таблицю незалежно від довжини полів. Три некоректні рядки не перервали обробку: вони потрапили до окремого списку з причиною відхилення.

Контрольні питання:

1. Як влаштована внутрішня модель рядків CPython відповідно до специфікації PEP 393? Чому рядок, що містить тільки ASCII-символи, займає 1 байт на символ, а поява одного символу кирилиці переводить весь рядок у 2-байтовий формат UCS-2?

За PEP 393 кожен об’єкт str зберігає символи в масиві фіксованої ширини, яку обирають за найбільшим кодовим пунктом рядка: 1 байт (кодові пункти до U+00FF), 2 байти (до U+FFFF) або 4 байти (понад U+FFFF). Рядок із самих ASCII-символів займає 1 байт на символ. Літери кирилиці мають коди U+0400 і вище, які не вміщуються в 1 байт, тому весь рядок переходить на 2 байти на символ. Єдина ширина для всіх символів потрібна, щоб доступ за індексом лишався O(1), без розбору змінної довжини, як в UTF-8.

2. Поясніть причину алгоритмічної неефективності конкатенації рядків у циклі через оператор += ($O(N^2)$) та обґрунтуйте лінійну складність O(N) методу str.join().

Рядки незмінні, тому кожна операція s += x в загальному випадку створює новий рядок і копіює в нього всі попередні символи. Для N кроків сумарна кількість копіювань пропорційна 1 + 2 + … + N, тобто $O(N^2)$. Метод join() спочатку обходить колекцію й обчислює сумарну довжину, потім один раз виділяє буфер потрібного розміру й копіює кожен фрагмент рівно один

раз, що дає $O(\Sigma L_i)$. (CPython іноді оптимізує +=, коли в рядка лише одне посилання, але ця поведінка не гарантується.)

3. У чому полягає різниця між методами регулярних виразів re.match(), re.search() та re.finditer()? У яких інженерних сценаріях доцільно використовувати finditer замість findall?

re.match() шукає збіг лише від початку рядка, re.search() — перше входження в будь-якому місці, re.finditer() повертає лінивий ітератор об'єктів Match для всіх непересічних входжень. findall() одразу будує список рядків або кортежів. finditer доцільний для великих текстів (результати генеруються по одному, без великого списку в пам'яті) і коли потрібні позиції збігів, span() чи groupdict(), а не тільки рядки.

4. Що таке «катастрофічне повернення» (catastrophic backtracking) у рушіях регулярних виразів NFA, і як правильне використання лінивих квантифікаторів запобігає зависанню програми?

Це експоненціальне зростання часу зіставлення в рушіях із поверненням, коли шаблон містить неоднозначні вкладені квантифікатори, наприклад (a+)+ $ на рядку aaaa…ab: рушій перебирає всі способи розбити послідовність a між вкладеними групами. Ліниві квантифікатори (+?, *?) лише змінюють порядок перебору й самі по собі проблеми не вирішують. Надійні способи: прибрати неоднозначність (різні символьні класи для сусідніх частин), використовувати якорі й вузькі класи замість .*, застосовувати атомарні групи та посесивні квантифікатори (у Python 3.11+), обмежувати довжину вхідних даних. У цій роботі шаблон побудований без вкладених квантифікаторів.

5. Опишіть синтаксис та призначення іменованих груп захоплення (?P<name>pattern) у регулярних виразах. Як отримати витягнуті дані у вигляді словника?

Конструкція (?P<name>pattern) захоплює підрядок, що збігся з pattern, і дає йому ім'я name. Доступ до значення — match.group("name"); усі іменовані групи одразу повертає match.groupdict() у вигляді словника {"name":

значення}. Для необов’язкових груп, що не брали участі в збігу, значенням буде None. Імена роблять код самодокументованим і не залежать від порядкових номерів груп, тому зміна шаблону не ламає решту коду.

6. Розкрийте переваги f-рядків (f-strings) над методом str.format() та оператором % з точки зору опкодів віртуальної машини PVM.

F-рядок розбирається під час компіляції, а вирази всередині фігурних дужок вбудовуються в байт-код безпосередньо: кожне значення форматується інструкцією FORMAT_VALUE (у Python 3.13 її замінили FORMAT_SIMPLE і FORMAT_WITH_SPEC), а BUILD_STRING збирає результат. Для str.format() під час виконання треба знайти метод, викликати його й розібрати рядок формату; оператор % створює проміжний кортеж аргументів і так само розбирає рядок формату. Тому f-рядки зазвичай швидші та читабельніші, а вираз стоїть поруч із місцем підстановки.